

IDOR / Broken Object Level Authorization (BOLA)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un IDOR (Insecure Direct Object Reference) ocurre cuando la app usa un identificador (un ID numerico, un UUID, un email) para acceder a un recurso, y el backend confía en que 'si conoces el ID, tienes permiso'. Si cambias el ID por el de otro usuario y el backend no revalida que ESE ID te pertenece, accedes a datos o acciones ajenas. BOLA es el mismo bug con nombre mas moderno, centrado en APIs.

El programador comprueba que ESTAS autenticado (tienes un token valido) pero se olvida de comprobar que ese token tiene permiso sobre ESE recurso concreto. Autenticacion (quien eres) y autorizacion (que puedes tocar) son cosas distintas, y este bug es exactamente confundirlas.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier endpoint que reciba un identificador de recurso: /api/users/{id}, /api/orders/{id}, /invoice/{id}.pdf, ?user_id=. Sospecha mas si los IDs son secuenciales (facil de adivinar) que si son UUIDs (aunque UUIDs filtrados en otra respuesta tambien sirven).

Cómo identificarla

- IDs numericos secuenciales en la URL o el JSON de respuesta.
- El frontend oculta botones/vistas segun el rol, pero la llamada a la API subyacente no cambia.
- Cambiar el ID devuelve 200 con datos que no son tuyos.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Con tu sesion, pide el mismo endpoint cambiando el ID por uno que sepas que no es tuyo (otro usuario, otro pedido).

Evidencia esperada: Un 200 con datos de otro usuario/recurso (o una accion aceptada sobre un recurso ajeno) confirma el IDOR. Un 403/404 descarta ese endpoint concreto.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
curl -sS "$URL/api/users/1001" -H "Authorization: Bearer $TOKEN"
```

Objetivo y contexto: Pide un recurso con un ID que probablemente no es el tuyo, usando tu propio token. Si el backend solo mira 'hay token valido' y no 'este token puede ver el 1001', el IDOR se confirma.

Resultado esperado: Datos de un usuario que no eres tu (nombre, email, direccion) devueltos con 200. Repite con varios IDs cercanos para confirmar el patron.

Anatomía de los payloads

La primera prueba es `curl -sS "$URL/api/users/1001" -H "Authorization: Bearer $TOKEN"`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un 200 con datos de otro usuario/recurso (o una acción aceptada sobre un recurso ajeno) confirma el IDOR. Un 403/404 descarta ese endpoint concreto.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Identifica todos los endpoints que reciben un ID de recurso (usuario, pedido, factura, mensaje).
- Crea o consigue dos identidades si el reto lo permite; compara la misma llamada con cada una.
- Cambia el ID de un recurso propio por uno ajeno y observa la respuesta completa, no solo el código HTTP.
- Prueba también acciones de escritura (PUT/DELETE) sobre IDs ajenos, no solo lectura (GET).

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El backend revalida en cada petición que el recurso pertenece al usuario autenticado (403/404 al cambiar el ID).
- El ID no identifica un recurso privado (es público por diseño, como el ID de un producto en un catálogo).
- Ejecutar la prueba sin adaptar variables ni versión.

- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Cambias el ID de un recurso en la URL o el JSON y accedes a datos o acciones de otro usuario porque el backend no revalida el permiso. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.

5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (idor-bola)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.